

DETECTION AND PREVENTION OF ATTACKS DURING FIRMWARE UPDATES IN IOT DEVICES USING MACHINE LEARNING AND DEEP LEARNING

By

MAMA M'BENGUE (mama@aims.edu.gh)

June 2026

*AN ESSAY PRESENTED TO AIMS-GHANA IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
MASTER OF SCIENCE IN MATHEMATICS*



AIMS

African Institute for
Mathematical Sciences
GHANA

DECLARATION

This work was carried out at AIMS-Ghana in partial fulfilment of the requirements for a Master of Science Degree.

I hereby declare that except where due acknowledgement is made, this work has never been presented wholly or in part for the award of a degree at AIMS-Ghana or any other University.

Student: MAMA M'BENGUE

A handwritten signature in black ink, appearing to read 'MAMA M'BENGUE', with a horizontal line drawn through it.

Supervisor: DAMIEN WOHWE SAMBO

A handwritten signature in blue ink, appearing to read 'DAMIEN WOHWE SAMBO', with a horizontal line drawn through it.

ACKNOWLEDGEMENTS

All praise is due to Allah, the Almighty, whose grace and mercy support me at every stage of my life. May peace and blessings be upon His Prophet Muhammad, a light and guide for all believers.

My deepest gratitude goes to AIMS, particularly AIMS Ghana, for opening the doors to an intellectual and human journey that I will never forget. This support has been far more than training; it has been a transformation.

I owe a great deal to my supervisor, **Dr.DAMIEN WOHWE SAMBO**, to whom I express my sincere gratitude. He not only provided me with a framework to carry out this thesis, but also a space of freedom where I was able to think, doubt, start over and grow. His kind rigor pushed me to go beyond myself, to question my certainties, to learn and then to relearn.

To my parents, **MAYABA M'BENGUE** and **AHOUMA LOU AHOU AGNÈS**, thank you for your unconditional love, your silent sacrifices and your unwavering support. This work belongs to you as much as it does to me.

To my older brothers, **M'BENGUE HAMADOU**, **KOUASSI ENSELME TRÉSOR** and **TIDIANE**, thank you for being there, simply and faithfully, in moments of doubt as well as in moments of joy.

To the **ZRO family** of Bingerville, your warm welcome and your boundless generosity have been a true home away from home. Even beyond these years, you continue to support me with the same kindness and I am deeply grateful to you.

I think with affection of the entire **AEEMCI** family of Nangui Abrogoua University, who accompanied and shaped me long before this journey began.

Finally, there is a thought I keep for the end, because it touches what is most intimate. My younger brother, **M'BENGUE MODOU FALL**, called back to Allah on June 24, 2025 and did not get to see the outcome of all my work. But it is by thinking of him that I keep working and it is to him that these pages are silently dedicated.

DEDICATION

To my younger brother, **M'BENGUE MODOU FALL** and to all people living with down syndrome (Trisomy 21). May Allah enfold them in his infinite mercy and grant them peace, strength and dignity.

List of Abbreviations

Abbreviation	Meaning
AUC	Area Under the ROC Curve
CNN	Convolutional Neural Network
CNN-LSTM	Convolutional Neural Network - Long Short-Term Memory
CPU	Central Processing Unit
CSV	Comma-Separated Values
DDoS	Distributed Denial-of-Service
DL	Deep Learning
DNS	Domain Name System
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
ICMP	Internet Control Message Protocol
IIoT	Industrial Internet of Things
IoT	Internet of Things
IP	Internet Protocol
KL	Kullback–Leibler
LSTM	Long Short-Term Memory
MITM	Man-In-The-Middle
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
NP	Neyman-Pearson
OOB	Out-Of-Bag
OTA	Over-The-Air
PCAP	Packet Capture
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RF	Random Forest
RFC	Request For Comments
ROC	Receiver Operating Characteristic
Sim	Simulated
SMOTE	Synthetic Minority Over-sampling Technique
SQL	Structured Query Language
SVM	Support Vector Machine
TCP	Transmission Control Protocol
TP	True Positive
TPR	True Positive Rate
UDP	User Datagram Protocol
XGBoost	Extreme Gradient Boosting

XSS

Cross-Site Scripting

Abstract

This thesis focuses on the detection and prevention of attacks that occur during OTA updates of IoT firmware, combining machine learning and deep learning approaches. The problem is formalized as a hypothesis testing framework within the Neyman-Pearson setting, allowing strict control of the false alarm rate. Five models are evaluated on the public Edge-IIoTset dataset; Random Forest, XGBoost, SVM, LSTM and CNN-LSTM. XGBoost stands out with the best overall performance, achieving an accuracy of 0.9676, an AUC of 0.9986 and an inference time of only 0.0135 ms per sample, making it fully suitable for resource-constrained IoT devices. The addition of an attention-based ensemble model that combines XGBoost, Random Forest and CNN-LSTM does not provide any significant additional improvement. In parallel, a realistic simulation framework was used to generate 3,600 enriched network sessions covering ARP, ICMP, DNS and MQTT protocols. When trained on this synthetic dataset, the model achieves an accuracy of 0.8619. Experiments combining synthetic data with Edge-IIoTset further show that it is possible to exceed 0.93 accuracy even when 80% of the data is simulated, highlighting the practical value of this approach. A four-level prevention mechanism, driven by model confidence, completes the proposed system. We conclude that this work demonstrates that a lightweight model such as XGBoost is sufficient to ensure real-time detection on constrained IoT devices and that adaptation to a new network environment remains feasible as long as a small locally labeled dataset is available.

Keywords: IoT, OTA firmware updates, Intrusion detection, Machine learning, Deep Learning, XGBoost, Network Simulation, Active Prevention, Neyman-Pearson.

Contents

Declaration	i
Acknowledgements	ii
Dedication	iii
List of Abbreviations	iv
Abstract	vi
1 Introduction	1
1.1 Problem Statement	1
1.2 Research Objectives and Contributions	1
1.3 Organisation of the Thesis	2
2 Literature Review	3
2.1 Security Threats in IoT OTA Firmware Updates	3
2.2 Machine Learning and Deep Learning for IoT Attack Detection	4
2.3 Machine Learning and Deep Learning for Firmware Security and OTA Updates	4
2.4 Positioning of the Present Work	5
3 Material and Methodology	6
3.1 Mathematical Formulation of Detection	6
3.2 Dataset: Edge-IIoTset	13
3.3 Performance Metrics	15
3.4 Experimental Workflow	16
3.5 Simulation Environment for Practical Validation	17
4 Results and Discussions	20
4.1 Overall Performance on Edge-IIoTset and Detection Time	20
4.2 Per-Class Analysis	20

4.3	Feature Importance	21
4.4	ROC Curves and AUC	21
4.5	Discussion of Edge-IIoTset Results	22
4.6	Model Performance on Simulated Data and Mixed Data	24
4.7	Active Prevention System	25
5	Conclusion and Perspectives	27
	References	29
	APPENDICES	29

1. Introduction

Every day, millions of connected devices such as; surveillance cameras, pacemakers, industrial valves silently download a new version of their firmware. In a few seconds a file crosses the network and installs itself at the heart of the device. This operation called Over-The-Air (OTA) update has become the universal reflex to remotely fix security flaws [5]. However, it hides a very large paradox where the mechanism designed to protect devices can itself become the source of its compromise.

If an attacker intercepts the communication, modifies the firmware in transit or forces the installation of an outdated and vulnerable version, it is the entire chain of trust that collapses even before the update is completed [10]. Traditional defense methods like digital signatures, hash fingerprints are not sufficient against a compromised server, a zero-day vulnerability or a similar attack [12]. It is not enough to only verify the integrity of the firmware; it is necessary to monitor in real time the behavior of the update process itself.

It is precisely here that Machine Learning (ML) and Deep Learning (DL) come into play. These techniques excel at detecting anomalies in complex data streams and distinguishing legitimate traffic from malicious attempts [1]. Applied to the OTA context, they offer the promise of near-instant detection, capable of blocking an attack before even a small trace is written onto memory. The present work explores this direction and addresses a question that is as simple as it is demanding, namely whether a machine learning model can both detect an OTA attack with sufficient accuracy and prevent it quickly from occurring.

1.1 Problem Statement

The deployment of OTA updates exposes IoT devices to numerous and diverse attacks (interception, modification, downgrade) [10]. Cryptographic mechanisms are not sufficient when the server is compromised or when a zero-day vulnerability is exploited [12]. While traffic analysis using ML/DL can detect these threats [13], it must be able to intervene before the firmware installation.

The problem is therefore to define which model offers the best trade-off between precision and detection speed, while validating its generalization on a second dataset obtained through a realistic multi-attack OTA simulation. To this end, the detection is modeled as a Neyman-Pearson hypothesis test [14] and evaluated using a public dataset complemented by the simulation.

1.2 Research Objectives and Contributions

The main objective is to build and validate a detection and prevention chain of attacks during IoT firmware OTA updates using ML/DL. A validation that is based on an experimentation first on the public dataset Edge-IIoTset and finally on the dataset produced by real simulation.

The contributions of our work are as follows: A structured review of OTA threats and ML/DL approaches for IoT firmware security. A mathematical formalization of the detection problem (hypothesis testing, metrics, model foundations). An experimental comparison of five models (Random Forest, XGBoost, SVM, LSTM, CNN-LSTM) on Edge-IIoTset with a measurement of detection time. The implementation of a real attack simulation generating a second dataset to test the generalization of the models. The integration of an active prevention mechanism (alert levels and resistance actions).

1.3 Organisation of the Thesis

Chapter 1 introduces the problem of OTA firmware updates and states the research objectives. Chapter 2 surveys the literature on OTA threats, ML/DL for IoT intrusion detection and firmware security. Chapter 3 establishes the mathematical formalisation of the detection problem and details the experimental methodology, including the Edge-IIoTset dataset, performance metrics and the simulation environment. Chapter 4 presents and discusses the results obtained on Edge-IIoTset and on the simulated OTA testbed, together with the active prevention mechanism. Finally, Chapter 5 summarises the contributions, discusses limitations and give perspectives for future work.

2. Literature Review

This chapter surveys the security threats specific to OTA firmware updates, reviews the ML and DL techniques that have been applied to attack detection in IoT networks and discusses existing solutions for firmware security. The chapter concludes by positioning the present work with respect to the identified gaps.

2.1 Security Threats in IoT OTA Firmware Updates

2.1.1 Update Process and Attack Surface.

An OTA update takes place in three phases:

- Discovery and download;
- Integrity and authenticity verification;
- Installation followed by reboot.

Each step presents a vulnerability such that channel interception, bypassing of security checks and malicious code injection[10]. These risks are amplified by IoT hardware constraints (CPU/RAM), which limit the use of heavy cryptographic protocols.

2.1.2 Threat Model.

In accordance with the model [10], three major attack vectors threaten the OTA process:

- **MITM:** The attacker intercepts the flow between the server and the device to modify or replace the firmware in transit.
- **Downgrade attack:** It forces the device to reinstall an older vulnerable version, often exploiting poor management of software freshness.
- **Malicious firmware injection:** After compromising the network or the server, the attacker deploys a corrupted firmware to integrate the device into a botnet.

The absence of strong security protocols makes early detection prior to installation immediately necessary. The network signatures of these attacks (traffic spikes, atypical packet sequences or checksum inconsistencies) constitute exploitable features for the machine learning approaches presented in the next Section 2.2.

2.2 Machine Learning and Deep Learning for IoT Attack Detection

2.2.1 Classical Supervised Algorithms.

Traditional machine learning algorithms have been widely applied to intrusion detection in IoT due to their effectiveness on tabular data. Several classifiers were compared for detecting MITM attacks in IoT networks and high performance was reported for Random Forest and XGBoost with accuracy exceeding 96% [1]. The effectiveness of these two models in a multiclass setting, together with the benefit of using feature importance to interpret decisions, was further confirmed [13]. SVM, although efficient on small datasets, does not scale well when the data volume increases [15].

2.2.2 Deep Sequential Models.

Recurrent neural networks and hybrid architectures are increasingly used for intrusion detection in IoT, as they capture hidden temporal dependencies in network traffic. A hybrid CNN-LSTM architecture achieving 99.96% multiclass accuracy on the CICIoT2023 dataset and 98.75% on TON_IoT was introduced by [16]. The authors highlight that this combination reduces the number of parameters compared to a pure LSTM while improving robustness. However, these deep models remain more computationally expensive in training time than decision-tree approaches.

2.2.3 Handling Imbalanced Data : SMOTE.

Intrusion detection datasets are highly imbalanced, with rare attacks often overshadowed by normal traffic. The SMOTE technique is the most widely used solution to mitigate this bias [3]. It generates synthetic samples of the minority class by linear interpolation between an example and its nearest neighbors. This avoids overfitting caused by duplication and expands the decision region of minority classes. Several recent IoT security studies combine SMOTE with tree-based models or deep networks, showing significant improvements in F1-score for underrepresented attack classes.

2.3 Machine Learning and Deep Learning for Firmware Security and OTA Updates

2.3.1 Malware Detection in Firmware.

Firmware analysis has been widely studied using ML. The application of ML (random forest, gradient boosting, logistic regression) and CNN to classify firmware as benign or malicious before installation [12]. Their study reports detection rates above 95% on real IoT firmware datasets. However, these approaches remain limited to static analysis, without exploiting network behavior during the update process.

2.3.2 Existing Solutions for Secure OTA.

The literature mainly proposes cryptographic mechanisms (digital signatures, hashing, trust chain verification) and standardized protocols such as the SUI framework (RFC 9019) to ensure firmware integrity and authenticity [5]. These preventive solutions are robust but do not cover attacks exploiting side channels, zero-day vulnerabilities or compromise of the legitimate server. Very few works integrate a real-time behavioral detection layer during the update session.

Reference	Methods	Dataset(s)	Contribution	Limitations
[10]	Static analysis, threat classification	23 commercial IoT devices	Identifies three OTA attack types (Modification Attack, Downgrade Attack, Brick Attack)	No real-time detection
[12]	RF, Gradient Boosting, CNN	IoT firmware binaries	ML/DL for malware detection with accuracy > 95%	Static only, no network behaviour during OTA
[16]	Hybrid CNN-LSTM + attention	CICIoT2023, TON_IoT	99.96% accuracy on binary and multiclass (CICIoT2023)	Not OTA-specific, high training cost
[1]	RF, XGBoost, SVM, LSTM	Simulated IoT network	MITM detection accuracy > 96%	Limited to MITM, no OTA context
[13]	XGBoost, Random Forest	CICIDS2017	Multiclass IDS with feature importance analysis	No OTA scenario, no temporal models
[5]	Cryptographic protocols	20 real IoT devices	Secure OTA testbed, standards-compliant	Preventive only, no behavioural detection
Our Work	XGBoost, attention ensemble, Neyman-Pearson, multi-attack simulation, active prevention	Edge-IIoTset + simulated dataset (3,600 sessions)	0.9676 accuracy, 0.9986 AUC, 0.0135 ms inference, local adaptation with 20% Edge-IIoTset data	Loopback simulation, MITM underrepresented

Table 2.1: Comparison of related works on IoT intrusion detection and OTA security.

2.4 Positioning of the Present Work

The above review reveals two major limitations which are : (1) Firmware security generally relies on static code analysis [12] and (2) OTA security solutions are essentially preventative [5]. No study has yet proposed a detection system capable of monitoring OTA transmission in real time using both classical and deep models. Our work is positioned at the intersection of IoT intrusion detection and update security:

- Analyze OTA session network data in real time.
- Compare detection efficiency and speed between tree-based models and neural networks.
- Validate the models on the public Edge-IIoTset dataset [7] and on a second dataset generated through a realistic local attack simulation.
- Integrate an active prevention system that triggers a response as soon as an attack is detected.

3. Material and Methodology

This chapter formalises the detection problem mathematically and describes the experimental pipeline. It introduces the NP framework, presents the Edge-IIoTset dataset and its preprocessing, defines the performance metrics, the experimental workflow and describes the simulation environment built for practical validation.

3.1 Mathematical Formulation of Detection

3.1.1 Binary Hypothesis Testing: Neyman-Pearson Framework.

Let $\mathbf{x} \in \mathbb{R}^d$ be a feature vector. The detection problem is formulated as a binary hypothesis test [14] given by Equation 3.1.1:

$$(\text{Normal}) H_0 : \mathbf{x} \sim p(\mathbf{x} | H_0) \quad , \quad (\text{Attack}) H_1 : \mathbf{x} \sim p(\mathbf{x} | H_1). \quad (3.1.1)$$

A decision rule is a function:

$$\delta : \mathbb{R}^d \rightarrow \{0, 1\}, \quad \delta(\mathbf{x}) = \begin{cases} 1 & H_1 \text{ (attack)} , \\ 0 & H_0 \text{ (normal)}. \end{cases} \quad (3.1.2)$$

Its error probabilities are:

$$P_{\text{FA}}(\delta) = P_{H_0}(\delta(\mathbf{x}) = 1), \quad P_{\text{MD}}(\delta) = P_{H_1}(\delta(\mathbf{x}) = 0). \quad (3.1.3)$$

The detection probability is:

$$P_D(\delta) = 1 - P_{\text{MD}}(\delta) = P_{H_1}(\delta(\mathbf{x}) = 1). \quad (3.1.4)$$

The NP problem seeks:

$$\delta^* = \arg \max_{\delta} P_D(\delta) \quad \text{subject to} \quad P_{\text{FA}}(\delta) \leq \alpha, \quad \alpha \in (0, 1). \quad (3.1.5)$$

If the distributions $p(\mathbf{x} | H_0)$ and $p(\mathbf{x} | H_1)$ are known, the optimal solution given by the NP Lemma [14] is the likelihood ratio test:

$$\lambda(\mathbf{x}) = \frac{p(\mathbf{x} | H_1)}{p(\mathbf{x} | H_0)}, \quad (3.1.6)$$

$$\delta^*(\mathbf{x}) = \begin{cases} 1 & \text{if } \lambda(\mathbf{x}) > \eta \\ 0 & \text{otherwise} \end{cases}, \quad \text{with } P_{H_0}(\lambda(\mathbf{x}) > \eta) = \alpha. \quad (3.1.7)$$

In practice, the densities are unknown. A classifier produces a score $s : \mathbb{R}^d \rightarrow [0, 1]$, yielding a parametrized family of decision rules:

$$\delta_t(\mathbf{x}) = \mathbf{1}_{\{s(\mathbf{x}) > t\}}, \quad t \in \mathbb{R}. \quad (3.1.8)$$

The ROC curve is defined as:

$$\{(P_{FA}(t), P_D(t)) : t \in \mathbb{R}\}. \quad (3.1.9)$$

The AUC equals:

$$\text{AUC} = \int_0^1 P_D(P_{FA}^{-1}(u)) du = P(s(\mathbf{x}_1) > s(\mathbf{x}_0)), \quad \mathbf{x}_1 \sim H_1 \text{ and } \mathbf{x}_0 \sim H_0, \quad (3.1.10)$$

which coincides with the Mann-Whitney U statistic [6]. Under the NP constraint, the optimal operating threshold is:

$$t^* = \sup\{t \in \mathbb{R} : P_{FA}(t) \leq \alpha\}. \quad (3.1.11)$$

The operational constraint requires the detection time to satisfy:

$$\Delta t_{\text{detection}} < \Delta t_{\text{OTA}}. \quad (3.1.12)$$

3.1.2 Anomaly Detection Criteria: Mahalanobis and KL Divergence.

In an OTA update, not all attacks are known in advance. We therefore learn only the normal behavior (H_0) and compute a statistic $T(\mathbf{x})$ that measures the deviation of an observation from this behavior. An alert is triggered if this deviation exceeds a threshold (τ):

$$\delta(\mathbf{x}) = \mathbf{1}_{\{T(\mathbf{x}) > \tau\}}. \quad (3.1.13)$$

This general principle applies both to the Mahalanobis distance and the KL divergence.

Mahalanobis Distance: To construct such a test, we first assume that normal traffic follows a multivariate Gaussian distribution:

$$H_0 : \mathbf{x} \sim \mathcal{N}(\mu, \Sigma), \quad (3.1.14)$$

which provides a complete probabilistic model of legitimate behavior. From a sample of normal sessions, the model parameters are estimated. The empirical mean,

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i, \quad (3.1.15)$$

gives the center of the normal data cloud, while the empirical covariance matrix,

$$\hat{\Sigma} = \frac{1}{n-1} \sum_{i=1}^n (\mathbf{x}_i - \hat{\mu})(\mathbf{x}_i - \hat{\mu})^\top, \quad (3.1.16)$$

captures the dispersion and correlations between features. Using these estimates, the Mahalanobis distance is computed for any new observation $\mathbf{x}$:

$$D_M^2(\mathbf{x}) = (\mathbf{x} - \hat{\mu})^\top \hat{\Sigma}^{-1} (\mathbf{x} - \hat{\mu}), \quad (3.1.17)$$

which measures the deviation from normality while accounting for variance in each direction.

Because the features are jointly Gaussian, the squared Mahalanobis distance follows a chi-square distribution under H_0 :

$$D_M^2(\mathbf{x}) \sim \chi^2(d) , \quad (3.1.18)$$

which allows setting a theoretical threshold controlling the false-alarm probability (α):

$$\tau^2 = \chi_{d, 1-\alpha}^2 . \quad (3.1.19)$$

The decision rule is then:

$$\delta(\mathbf{x}) = \mathbf{1}_{\{D_M^2(\mathbf{x}) > \tau^2\}} . \quad (3.1.20)$$

An alarm is triggered whenever the normalized distance exceeds the threshold. Geometrically, using the Cholesky factorization $\hat{\Sigma} = LL^\top$, one can whiten the data, turning the distance into a Euclidean norm:

$$D_M^2(\mathbf{x}) = \|L^{-1}(\mathbf{x} - \hat{\mu})\|_2^2 , \quad (3.1.21)$$

showing that Mahalanobis-based detection is equivalent to thresholding the Euclidean norm after whitening.

KL Divergence: The Gaussian assumption may be too restrictive. An alternative is to directly compare the observed traffic distribution with a reference distribution Q learned from normal traffic. We first estimate the empirical distribution $\hat{P}$ over a sliding window of m samples:

$$\hat{P}(x) = \frac{1}{m} \sum_{j=1}^m \mathbf{1}_{\{x_j=x\}} . \quad (3.1.22)$$

The KL divergence:

$$D_{KL}(\hat{P}||Q) = \sum_{x \in \mathcal{X}} \hat{P}(x) \log \left(\frac{\hat{P}(x)}{Q(x)} \right) , \quad (3.1.23)$$

quantifies the difference between the observed and the normal distribution. A continuous version exists for densities:

$$D_{KL}(p||q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx . \quad (3.1.24)$$

The decision rule keeps the same structure, using the KL divergence as the test statistic:

$$\delta = \mathbf{1}_{\{D_{KL}(\hat{P}||Q) > \tau\}} , \quad (3.1.25)$$

where τ is chosen to control false alarms. For small deviations, the quadratic approximation given by Equation 3.1.26 relates KL divergence to a weighted distance, facilitating threshold tuning.

$$D_{KL}(P||Q) \approx \frac{1}{2} \sum_x \frac{(P(x) - Q(x))^2}{Q(x)} , \quad (3.1.26)$$

3.1.3 ROC Curve and AUC.

For classification, an attack can be predicted if its score exceeds a threshold t . The two key measures that follow are:

$$\text{TPR}(t) = P(s(x) > t \mid H_1) \quad (\text{True Positive Rate}), \quad (3.1.27)$$

$$\text{FPR}(t) = P(s(x) > t \mid H_0) \quad (\text{False Positive Rate}). \quad (3.1.28)$$

The ROC curve plots the relationship between successful detections and false alarms for all possible thresholds. The AUC summarizes this performance with a single value between 0 and 1.

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(u)) du. \quad (3.1.29)$$

The AUC measures the probability that the model ranks a positive example higher than a negative one:

$$\text{AUC} = P(s(x_1) > s(x_0) \mid x_1 \sim H_1, x_0 \sim H_0), \quad (\text{Mann - Whitney } U \text{ statistic}). \quad (3.1.30)$$

In the NP framework, the optimal threshold maximizes the true positive rate (TPR) while constraining the false positive rate (FPR) to a level α . The ROC curve and AUC therefore allow comparison of models independently of any predefined threshold.

3.1.4 Key Model Equations.

Random Forest combines multiple trees trained on different bootstrap samples. To reduce correlation, each node selects the best split criterion only from a random subset of features [2].

Node impurity is measured by the Gini index:

$$\text{Gini}(\mathcal{D}) = \sum_{k=1}^K \hat{p}_k(1 - \hat{p}_k) = 1 - \sum_{k=1}^K \hat{p}_k^2, \quad (3.1.31)$$

where $\hat{p}_k$ is the proportion of samples of class k in the node. The chosen split is the one that minimizes this impurity. The final prediction is obtained by Equation 3.1.32:

$$\hat{y} = \arg \max_k \sum_{b=1}^B \mathbf{1}\{T_b(\mathbf{x}) = k\}. \quad (3.1.32)$$

Bagging reduces the overall model variance by Equation 3.1.33:

$$\text{Var}(\hat{y}) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2, \quad (3.1.33)$$

where σ^2 is the variance of a single tree and ρ is the correlation between two trees. Increasing the number of trees B reduces the residual error, while random feature selection reduces the correlation ρ between trees.

The OOB error uses samples excluded from bootstrap sampling to estimate model performance without requiring a separate validation set [2].

XGBoost builds decision trees sequentially to progressively correct the errors of previous trees and improve OTA attack detection [4].

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

This objective function defines the global loss at iteration t , combining the error between the true label y_i and the current prediction and the contribution of the new tree f_t .

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda |w|^2 \quad (3.1.34)$$

This term regularizes the model by penalizing the number of leaves T and the magnitude of weights w in order to limit tree complexity. The loss around $\hat{y}_i^{(t-1)}$ is approximated as:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t). \quad (3.1.35)$$

This transforms the problem into a quadratic form to simplify optimization:

$$g_i = \partial_{\hat{y}} l(y_i, \hat{y}_i^{(t-1)}), \quad h_i = \partial_{\hat{y}}^2 l(y_i, \hat{y}_i^{(t-1)}) , \quad (3.1.36)$$

these terms represent respectively the direction of error correction and its local sensitivity, enabling model updates. The optimal weight of each leaf is:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} , \quad (3.1.37)$$

this gives the optimal weight w_j of leaf j , where I_j is the set of samples in that leaf.

The quality of a split is measured by the Gain, which represents the reduction of the regularised loss achieved by splitting the node:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma . \quad (3.1.38)$$

This criterion measures the improvement obtained by splitting a node into left and right branches. (G_L, G_R) are sums of gradients and (H_L, H_R) are sums of Hessians in each subgroup.

Splits are chosen to maximize this gain in order to build more efficient trees. γ penalizes unnecessary splits to avoid overly complex trees, λ stabilizes weights and reduces model variance, the learning rate η reduces the contribution of each tree to make learning more stable.

Subsampling of data and features introduces diversity between trees reduces correlation and improves overall robustness. In the OTA context, this process progressively distinguishes normal traffic from complex malicious behavior.

LSTM networks are designed to capture temporal dependencies in sequences while avoiding the vanishing gradient problem that affects classical RNNs [9]. At each time step t , the unit receives the current input x_t and the previous hidden state h_{t-1} to update its internal memory. The first operation computes the forget gate :

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad (3.1.39)$$

which determines what part of the previous cell state C_{t-1} should be retained.

Next, the input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

decides which new information should be stored, while a candidate vector:

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

proposes possible new values to add. The cell state is then updated by combining the retained past information and the newly gated candidate:

$$C_t = f_t \circ C_{t-1} + i_t \circ \tilde{C}_t,$$

where $\circ$ denotes element-wise multiplication. Finally, the output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o),$$

controls how much of the filtered cell state is exposed as the hidden output:

$$h_t = o_t \circ \tanh(C_t).$$

Because the cell state update is additive, the gradient can flow across many time steps without vanishing, allowing the LSTM to learn long-range dependencies. This property makes it well suited for modelling the temporal relationships present in OTA packet sequences.

In OTA data, packets exhibit temporal relationships (bursts, anomalies). The LSTM models these dynamics to detect attacks. LSTM layers followed by dense layers enable final classification.

CNN-LSTM architecture combines local pattern extraction via 1D convolution and temporal modeling via LSTM for OTA sequence analysis [16]. We consider a sequence $x \in \mathbb{R}^{T \times d}$, where T is the temporal length and d is the number of features.

The 1D convolution is defined as:

$$(s * k)(t) = \sum_{\tau=1}^m k(\tau) \cdot x(t + \tau - 1)$$

where $k \in \mathbb{R}^m$ is a filter of size m and $s(t)$ is the local output. This operation extracts local patterns within the sequence.

The use of multiple filters (64, 128, 256) allows learning increasingly abstract representations. Each convolutional layer is followed by batch normalization to stabilize training and dropout to reduce overfitting.

The LSTM captures long-term temporal dependencies in the sequence. Its final hidden state h_T summarizes the global temporal information. This representation is passed to fully connected layers for classification. The final output is:

$$\hat{y} = \text{softmax}(Wh_T + b),$$

where W is the weight matrix and b the bias. Softmax converts scores into probabilities $\hat{y}$ the class probabilities.

Thus, the CNN extracts local patterns while the LSTM models their temporal evolution for OTA classification.

SMOTE

In anomaly detection, the rarity of attacks biases models in favor of normal traffic. SMOTE corrects this imbalance by creating synthetic attack examples. For each real attack, the algorithm generates a new point by interpolation between it and one of its nearest neighbors. For each minority example x , one of its k nearest neighbors x_{nn} (based on Euclidean distance) is randomly selected. A new example x_{new} is created as follows:

$$x_{new} = x + \lambda \cdot (x_{nn} - x), \quad \lambda \sim \mathcal{U}(0, 1).$$

SMOTE balances the classes by creating new unique data rather than simple copies, which limits overfitting. Geometrically, it expands the attack detection region. The risk of generating unrealistic data is controlled by tuning the number of neighbors k . In our study, SMOTE is applied only to the training set.

Adam Optimiser

Training of neural networks (LSTM, CNN-LSTM) uses the Adam optimizer [11].

At each iteration t , the gradient of the loss is computed:

$$g_t = \nabla_{\theta} \mathcal{L}(\theta_{t-1}).$$

Adam builds two moving averages. The first tracks the average direction of the gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

The second measures the magnitude of gradient variations:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

These estimates are corrected for initial bias:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The parameters are then updated as:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t,$$

where η is the learning rate and ϵ ensures numerical stability.

Thus, m_t stabilizes the descent direction, while v_t automatically adapts the step size to avoid oscillations. Adam's regret is bounded in the convex case, making it a standard optimizer for deep models due to its robustness to noisy gradients and fast convergence [11].

3.2 Dataset: Edge-IIoTset

This section presents the overall architecture of our attack detection and prevention system. Figure 3.1 details each step, from data preparation and model training to attack detection and the implementation of automated security responses. The goal is to effectively block attacks in real time.

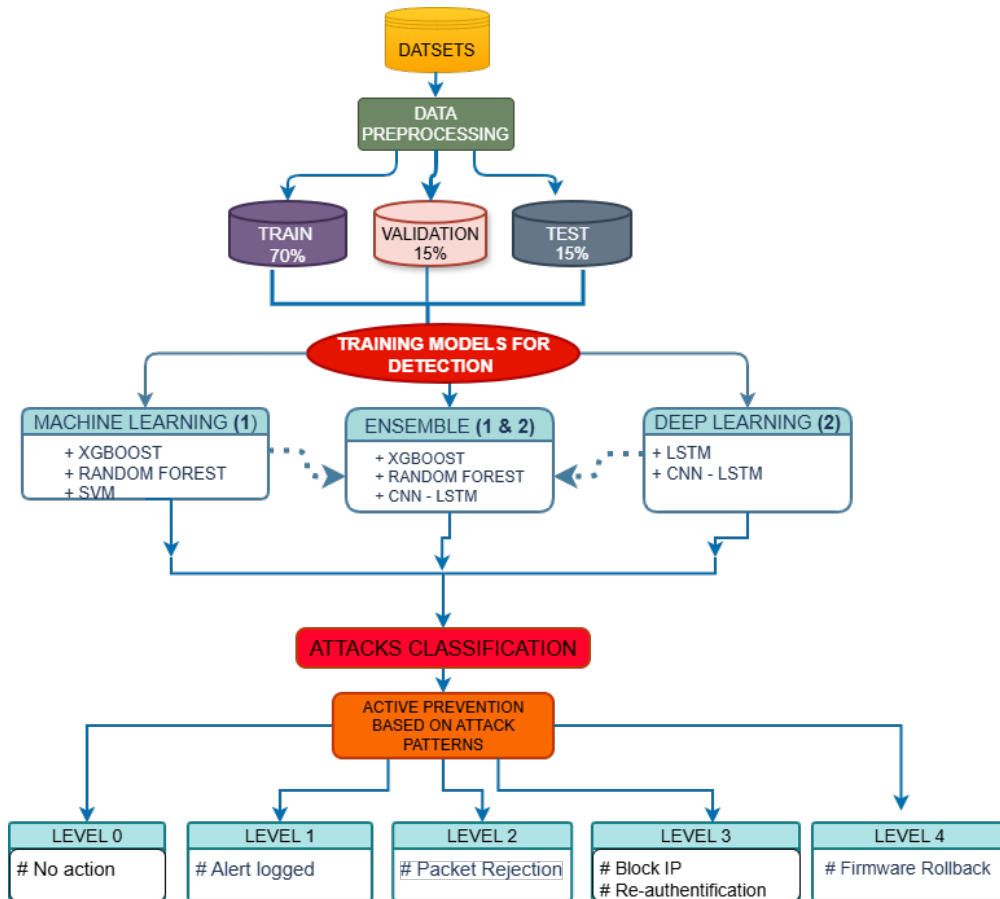


Figure 3.1: Global pipeline of the system for detecting and responding to attacks.

3.2.1 Origin, Cleaning, Features and Attack Classes.

We use the public Edge-IIoTset dataset [7], designed for cybersecurity in IoT/IloT environments. It provides multi-layer network traffic (sensors, edge, cloud) with both normal sessions and 14 types of attacks. The raw dataset is a matrix $\mathbf{X} \in \mathbb{R}^{157,800 \times 63}$ with no missing values observed.

We detect and remove 814 duplicate rows. Non-numerical columns are also excluded from the analysis. The cleaning process produces identical results every time and we obtain $\mathbf{X}_{\text{clean}} \in \mathbb{R}^{152,371 \times 57}$.

3.2.2 Class Grouping and Feature Reduction.

The 15 initial classes are imbalanced. To avoid fragmentation of rare samples and stabilize learning, we merge functionally similar attacks into 6 class defined by [7]:

- **(1) Normal:** legitimate network traffic without malicious activity.
- **(2) DDoS:** distributed denial-of-service attacks intended to overwhelm the OTA server through SYN, UDP, HTTP or ICMP flooding.
- **(3) Intrusion:** malicious attempts such as SQL injection, XSS attacks, unauthorised access, password theft or ransomware deployment.
- **(4) Backdoor:** installation of hidden remote access mechanisms or exploitation of known vulnerabilities.
- **(5) Reconnaissance:** information-gathering activities including port scanning and operating system fingerprinting.
- **(6) MITM:** man-in-the-middle attacks involving interception or modification of firmware during transmission.

The resulting class distribution is presented in table 3.1.

Grouped class	Merged attacks	Count
DDoS	DDoS_UDP, DDoS_ICMP, DDoS_HTTP, DDoS_TCP	48 399
Intrusion	SQL_injection, Uploading, Password, XSS, Ransomware	29 808
Backdoor	Backdoor, Vulnerability_scanner	29 788
Reconnaissance	Port_Scanning, Fingerprinting	19 830
MITM	MITM	394
Normal	Normal	24 152

Table 3.1: Table grouping attacks by different classes.

The grouping in Table 3.1 is justified by the similarity of network signatures and is consistent with the recommendations of [7]. It also covers the main OTA attack vectors identified by [10] described in the Section 2.1.2. The `Attack_type` column (classification target) is separated from the dataset after class grouping. We then obtain 56 numerical descriptive attributes.

Let $\mathbf{X}' \in \mathbb{R}^{152,371 \times 56}$ be this matrix. We compute the empirical variance of each column given by Equation 3.2.1

$$\sigma_j^2 = \frac{1}{n-1} \sum_{i=1}^n (X'_{ij} - \bar{X}'_j)^2, \quad j = 1, \dots, 56. \quad (3.2.1)$$

A Variance threshold filter is applied. Thus, any column satisfying $\sigma_j^2 = 0$ is removed since a constant carries no discriminative information. Therefore, remove 18 features and retaining the remaining 38 columns ($d = 38$).

3.2.3 SMOTE Resampling and Stratified Split.

The MITM class, with 394 samples, represents less than 0.26% of the dataset, making the data highly imbalanced and potentially biased toward majority classes.

To address this issue, smote is applied only on the training set [3]. It generates synthetic samples by interpolating between a minority sample $\mathbf{x}$ and one of its nearest neighbors.

This process is repeated until all classes are balanced in the training set. After applying, the dataset is split into stratified subsets: training (167 394 sample), validation(18 826 samples) and test (45 712 samples).

3.3 Performance Metrics

The evaluation of the models is based on the metrics defined in Chapter 3. We recall here their operational expressions, then we detail the measurement of detection time.

3.3.1 Standard Metrics (Precision, Recall, F1-score, AUC).

Let the test set be $\mathcal{T} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ where $\mathbf{x}_i$ is the feature vector of the traffic and y_i is the associated class label. For each class k , we denote TP_k , FP_k and FN_k respectively the true positives, false positives and false negatives. We define:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \quad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}. \quad (3.3.1)$$

Also, the macro and weighted versions of the F1-score are obtained by averaging the $F1_k$ across classes without or with weighting by the support of each class.

The AUC is computed as the probability that the score of a positive example exceeds that of a negative example [6]. It should be noted that all these indicators are reported on the test set.

3.3.2 Detection Time(Inference per Sample).

In an OTA update context, detecting an attack is only valuable if it occurs before the attack effects take place. We therefore measure the inference time per sample, defined as the time elapsed between submitting the vector $\mathbf{x}$ to the model and obtaining the decision $\delta(\mathbf{x})$.

Let t_i be the inference time for sample i with $t_i = t_{\text{end}} - t_{\text{start}}$. The average inference time on the test set is:

$$\bar{t}_{\text{inf}} = \frac{1}{N} \sum_{i=1}^N t_i. \quad (3.3.2)$$

This metric is measured under the same hardware conditions for all models. It is central to model comparison because a highly accurate but too slow model will not be very useful in practice. We will simultaneously analyze accuracy and AUC in order to assess the trade-off between precision and speed.

3.4 Experimental Workflow

3.4.1 Data Splits and SMOTE Application.

The cleaned data are divided into three disjoint subsets:

- **Training (70%)**: used to learn the model parameters.
- **Validation (15%)**: used to tune hyperparameters and reduce overfitting.
- **Test (15%)**: used to evaluate the final performance on unseen data.

Stratification ensures that the class distribution is preserved in each subset before applying SMOTE. SMOTE is then applied, producing a total of 27,899 balanced samples. The validation and test sets do not undergo any resampling, thus ensuring an unbiased evaluation.

3.4.2 Trained Models and Hyperparameters.

The following models are implemented and compared in this study:

- **Random Forest**:
`n_estimators=200, max_depth=None, min_samples_split=2, min_samples_leaf=1, random_state=42, n_jobs=-1`
- **XGBoost**: `n_estimators = 300, max_depth = 6, learning_rate = 0.1, subsample = 0.8, colsample_bytree = 0.8, reg_lambda = 1.0, reg_alpha = 0.0, random_state = 42, n_jobs = -1, eval_metric = mlogloss, verbosity = 0.`
- **SVM**: `kernel = rbf, C = 10, gamma = scale.`
- **LSTM**: `LSTM(256) → LSTM(128) → LSTM(64) → DENSE(64) → DENSE(N_CLASSES, SOFTMAX).` DROPOUT RATES: 0.3, 0.3, 0.2. ADAM OPTIMIZER (LEARNING_RATE = 0.001), LOSS = CATEGORICAL_CROSSENTROPY, EPOCHS = 50, BATCH_SIZE = 128.
- **CNN-LSTM**: `Conv1D(64) → Conv1D(128) → Conv1D(256) → LSTM(128) → Dense(128) → Dense(64) → Dense(n_classes, Softmax).` BatchNormalization after Conv1D and LSTM layers. Dropout rates: 0.2, 0.2, 0.2, 0.3, 0.2. L2 regularization (0.001) on LSTM and Dense(128). Adam optimizer (learning_rate = 0.001), loss = categorical_crossentropy, epochs = 50, batch_size = 128.

- **Ensemble (Attention):** an attention module composed of two dense layers (64 neurons with ReLU), followed by a 3-output softmax, learns to dynamically weight the predictions of RF, XGBoost, and CNN-LSTM. The base models are pre-trained and remain unchanged; only the attention layer is trained on the validation set.

3.4.3 Implementation Environment.

All experiments were conducted in a reproducible Python environment. The computational setup consisted of a Windows 11 operating system, an Intel Core i7 processor, 16 GB of RAM and an Intel UHD Graphics 620 integrated GPU. Python 3.13 was used throughout the study. Traditional machine learning classifiers, including Random Forest, Support Vector Machine (SVM) and XGBoost, were implemented using `scikit-learn` and `xgboost`. The deep learning models, such as LSTM, CNN-LSTM and the attention module, were developed with TensorFlow. Data preprocessing and analysis were performed using `pandas` and `NumPy`, while the SMOTE technique for class imbalance handling was implemented with the `imbalanced-learn` library.

3.5 Simulation Environment for Practical Validation

In order to validate the selected XGBoost model on Edge-IIoTset in a more realistic environment, a controlled simulation of OTA updates was designed. The entire testbed, attack scenarios and data collection were implemented from scratch by us. This simulation made it possible to generate a second dataset, independent of the first, which was then mixed with Edge-IIoTset in different proportions to assess the model's adaptability.

3.5.1 Justification and Objectives.

This work aims to verify whether the results obtained with the Edge-IIoTset dataset remain valid on a second independently created dataset. To this end, a simulation is set up to reproduce a real OTA update between a server and an IoT device. The simulation includes normal traffic enriched with five types of attacks (DDoS, Injection, Backdoor, Reconnaissance and MITM).

The objective is to generate a new labeled dataset in order to test the already trained XGBoost model without retraining it. It should be noted that the simulation incorporates additional network protocols such as ARP, ICMP, DNS and MQTT to better approximate a real IoT environment and make the data more comprehensive [7].

3.5.2 Architecture and Attack Scenarios.

The testbed is fully deployed on a single windows 11 machine using the loopback interface (127.0.0.1). All components (server, client, attack tools and network capture) run locally, without any external hardware infrastructure. The system is composed of three main elements: OTA Server, IoT Client and Attack Tools.

OTA Server: A Flask application provides a simulated binary firmware via HTTP on port 5000. A second instance on port 5001 serves a modified firmware used for the Backdoor scenario.

IoT Client: Python scripts simulate an IoT device that periodically queries the server to download updates. According to the scenario:

port 5000: normal traffic, DDoS, Intrusion, Reconnaissance

port 5001: Backdoor scenario

port 8080: MITM scenario via proxy

Attack Tools: Several tools and Python scripts are used to simulate attacks:

Scapy for DDoS attacks (SYN flood), mitmproxy to intercept and modify firmware (MITM), nmap for port reconnaissance, python scripts for malicious request injection (Intrusion) and backdoor server activation.

Protocol Enrichment

To reproduce more realistic network traffic, background traffic is injected before each capture session. It includes ARP, ICMP, DNS and MQTT exchanges generated in a simulated manner. This step ensures the completeness of the 38 extracted features and avoids null values in certain columns.

Experimental Scenarios

Six scenarios are defined. Each corresponds to a controlled sequence of interactions between the IoT client, the server and possibly an attack agent [10]. Each scenario is repeated 600 times, resulting in a total of 3600 sessions.

Normal: The IoT client periodically sends an HTTP request to the OTA server (port 5000). The server responds with a valid firmware, which is downloaded without modification or interruption. No malicious traffic is injected during the session.

DDoS: While the client performs a firmware update request, a Scapy-based script generates a massive flow of TCP SYN packets targeting the OTA server. This traffic overloads the server connection queue, leading to degradation or delayed responses to legitimate client requests.

Intrusion: The client sends an HTTP request containing a malicious payload in the form of SQL injection. This request is intended to simulate an attempt to exploit an application vulnerability. The server receives and processes the request as a standard HTTP flow, but the request content is marked as malicious in the dataset.

Backdoor: The IoT client is redirected to an alternative server (port 5001) simulating a compromised server. This server provides a modified firmware. The download proceeds normally from the client's perspective, but the transferred content originates from an unauthorized source.

Reconnaissance: Before any update, an external agent performs a port scan using nmap on the client or server address. This scan generates a series of requests aimed at identifying active services, open ports and possible service versions.

MITM: Traffic between the client and the server is intercepted via mitmproxy. During firmware transfer, the proxy acts as an intermediary and modifies the packet content before it reaches the client. Thus, the client believes it receives a legitimate firmware, while it has been altered in

transit.

3.5.3 Data Collection and Feature Extraction.

The 600 sessions were captured for each attack class using Tshark on the loopback interface (127.0.0.1). The obtained files are processed as those of the Edge-IloTset dataset. The 38 raw fields extracted from each .pcap file using Tshark were aggregated by session (sum, average or first value) to generate the rows of the `simulation_enriched.csv` dataset, which is directly compatible with the model trained on Edge-IloTset.

Component	Description
Platform	Single windows 11 machine, loopback interface (127.0.0.1)
OTA server	Flask application serving a dummy firmware on port 5000 and second instance on port 5001 for backdoor scenario
IoT client	Python scripts periodically fetching the firmware (ports 5000, 5001, 8080)
Attack tools	Scapy (SYN flood for DDoS), mitmproxy (MITM firmware modification), nmap (port scan for reconnaissance), python scripts (SQL injection for intrusion, backdoor server activation)
Background traffic injection	ARP, ICMP, DNS (scapy and socket scripts), MQTT (mosquitto broker + paho-mqtt publisher)
Network capture	Tshark on Npcap loopback adapter, one .pcap file per session
Number of sessions	6 scenarios $\times$ 600 repetitions = 3600 labelled sessions
Feature extraction	38 fields extracted via Tshark, aggregated per session (sum, mean, first value) into <code>simulation_enriched.csv</code>

Table 3.2: Simulation testbed configuration.

4. Results and Discussions

4.1 Overall Performance on Edge-IIoTset and Detection Time

Model	Acc.	F1	AUC	Train (s)	Infer. (ms)	Acc./Inf.
Random Forest	0.9600	0.9604	0.9972	13.44	0.0089	107.87
SVM	0.6663	0.6709	0.9242	13282.35	18.1896	0.04
XGBoost	0.9676	0.9679	0.9986	32.67	0.0135	71.67
LSTM	0.9150	0.9162	0.9908	1122.65	0.1377	6.65
CNN-LSTM	0.9157	0.9168	0.9906	13675.07	0.7665	1.19
Ensemble	0.9676	0.9679	0.9986	9.90	0.0540	17.92

Table 4.1: Performance comparison of different models.

Table 4.1 Random Forest has the highest ratio of 93.9, thanks to very fast inference. XGBoost achieves a ratio of 57.5, but remains the most performant model with the best accuracy of 0.9659 and the best AUC of 0.9985. This makes it the best trade-off choice for real-time use, as it detects attacks very effectively while remaining fast. Deep learning models (LSTM, CNN-LSTM) and the ensemble approach are less efficient, both in terms of accuracy and speed.

4.2 Per-Class Analysis

Model	Normal	Backdoor	DDoS	Intrusion	MITM	Recon.
Random Forest	1.00	0.96	0.96	0.91	1.00	0.99
SVM	0.71	0.59	0.78	0.48	0.70	0.76
XGBoost	1.00	0.97	0.97	0.93	1.00	0.99
LSTM	1.00	0.91	0.92	0.83	1.00	0.95
CNN-LSTM	1.00	0.91	0.92	0.83	1.00	0.94
Ensemble	1.00	0.97	0.97	0.93	1.00	0.99

Table 4.2: F1-score comparison by attack.

Table 4.2 shows that Normal traffic is correctly classified by all models, reducing the risk of unnecessary interruption of legitimate updates. MITM attacks also achieve a perfect F1-score, although their limited number of samples requires further validation through simulation. The Intrusion class remains the most difficult to detect, where XGBoost and the ensemble model obtain the best F1-score (0.93), while SVM performs poorly (0.48).

4.3 Feature Importance

Figure 4.1 shows that `tcp.options` is the most important variable (74,000). Then, `dns.qry.name.len` (28,000) and `dns.qry.qu` (22,000) also play a major role, followed by some TCP fields (between 18,000 and 5,000). All other variables (UDP, ICMP, HTTP, MQTT, ports and other DNS fields) have a low impact less or equal than 4,000. The remark is that the model mainly decides based on TCP options and DNS queries.

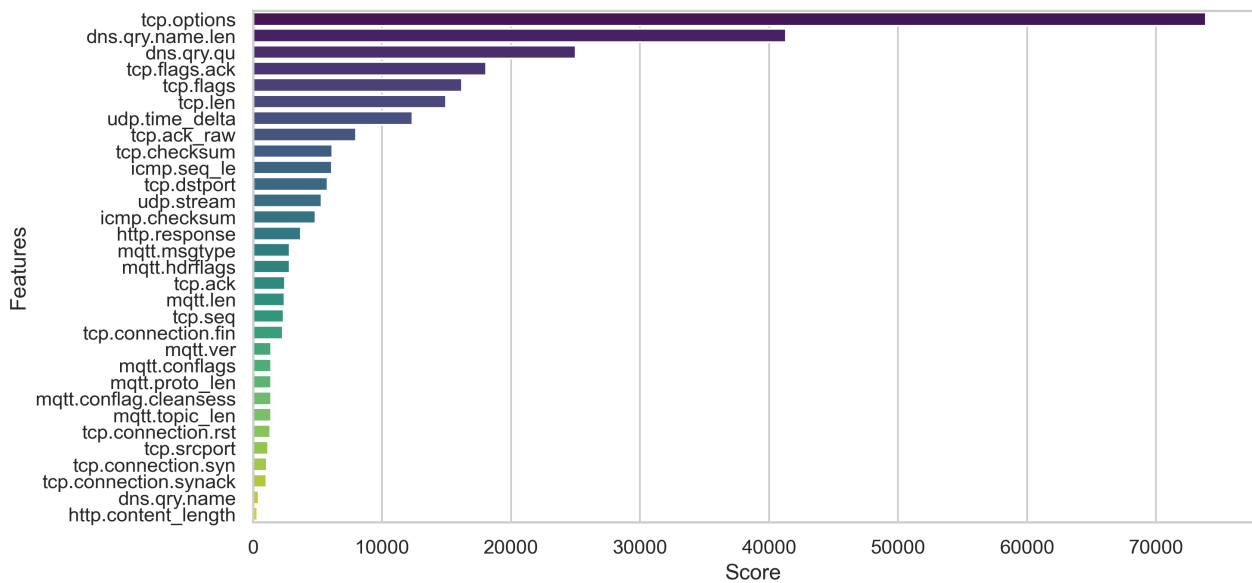


Figure 4.1: Importance of Features for Network Anomaly Detection.

4.4 ROC Curves and AUC

The Figure 4.2 shows the ROC curves of the different models. The AUC measures the ability of a model to correctly distinguish attacks from normal traffic. The results show that all models except SVM perform very well, with an AUC higher than 0.98. XGBoost and Random Forest are the best models with values of 0.9985 and 0.9971. This means that they detect almost all attacks with very few false alarms.

The ensemble model is also highly effective with an AUC of 0.9978. The SVM is the least performing model with an AUC of 0.9145.

Overall, these results show that the models are able to clearly separate the data and confirm the effectiveness of the chosen approach.

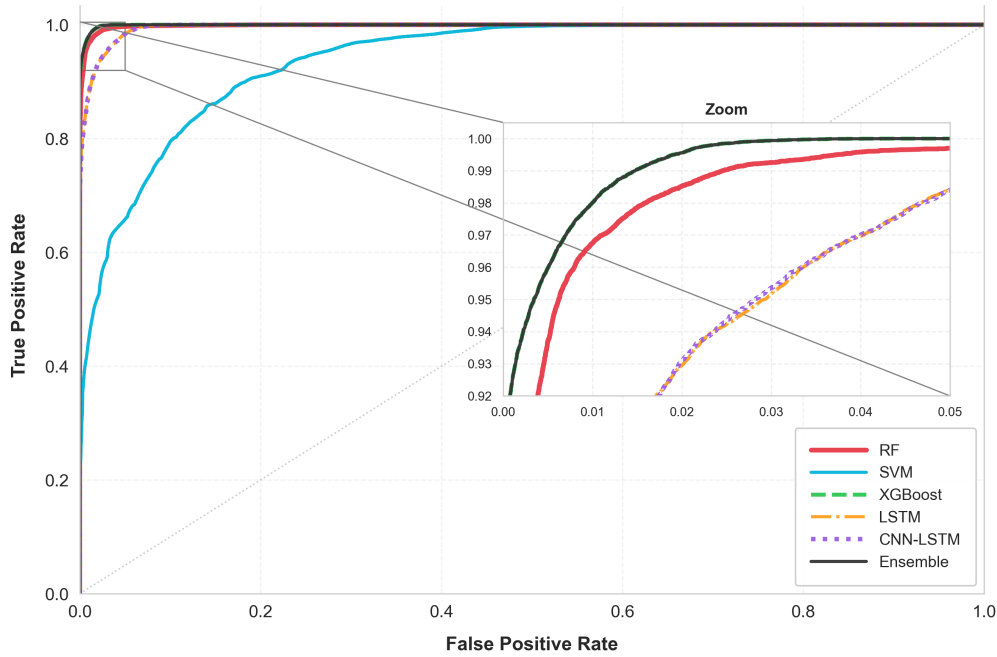


Figure 4.2: ROC curves of the different models on Edge-IIoTset.

4.5 Discussion of Edge-IIoTset Results

Based on our analyses, **XGBoost** clearly emerges as the best model on this dataset. With an accuracy of 0.9676, an F1-score of 0.9679 and an AUC of 0.9986, it outperforms all other tested models while being nearly 57 times faster than CNN-LSTM (0.0135 ms versus 0.7665 ms). Random Forest is slightly less accurate but predicts even faster, making it a strong alternative when latency is the main constraint.

The attention-based ensemble produces exactly the same predictions as XGBoost alone (accuracy 0.9676, F1-score 0.9679, AUC 0.9986), with higher complexity and latency. It therefore offers no measurable improvement and is not retained.

This result is consistent with the findings of [8], who showed that boosted tree models such as XGBoost perform extremely well on medium-sized tabular data. The 38 features of Edge-IIoTset correspond exactly to this type of data, consisting of aggregated flow statistics without spatial or temporal structure. Deep learning models, designed for raw data, are less suitable here and are further affected by class imbalance.

At the class level, two results stand out. First, **normal traffic** and **MITM** attacks are detected without error, which is essential for an OTA system. In contrast, the **Intrusion** class remains the most difficult, with an F1-score of 0.93. This class is highly heterogeneous, preventing the model from learning a consistent decision boundary. Figure 4.3 shows the network patterns behind these class-specific behaviors.

Figure 4.3 shows that the **DDoS** traffic shows a stable plateau between approximately 18 and

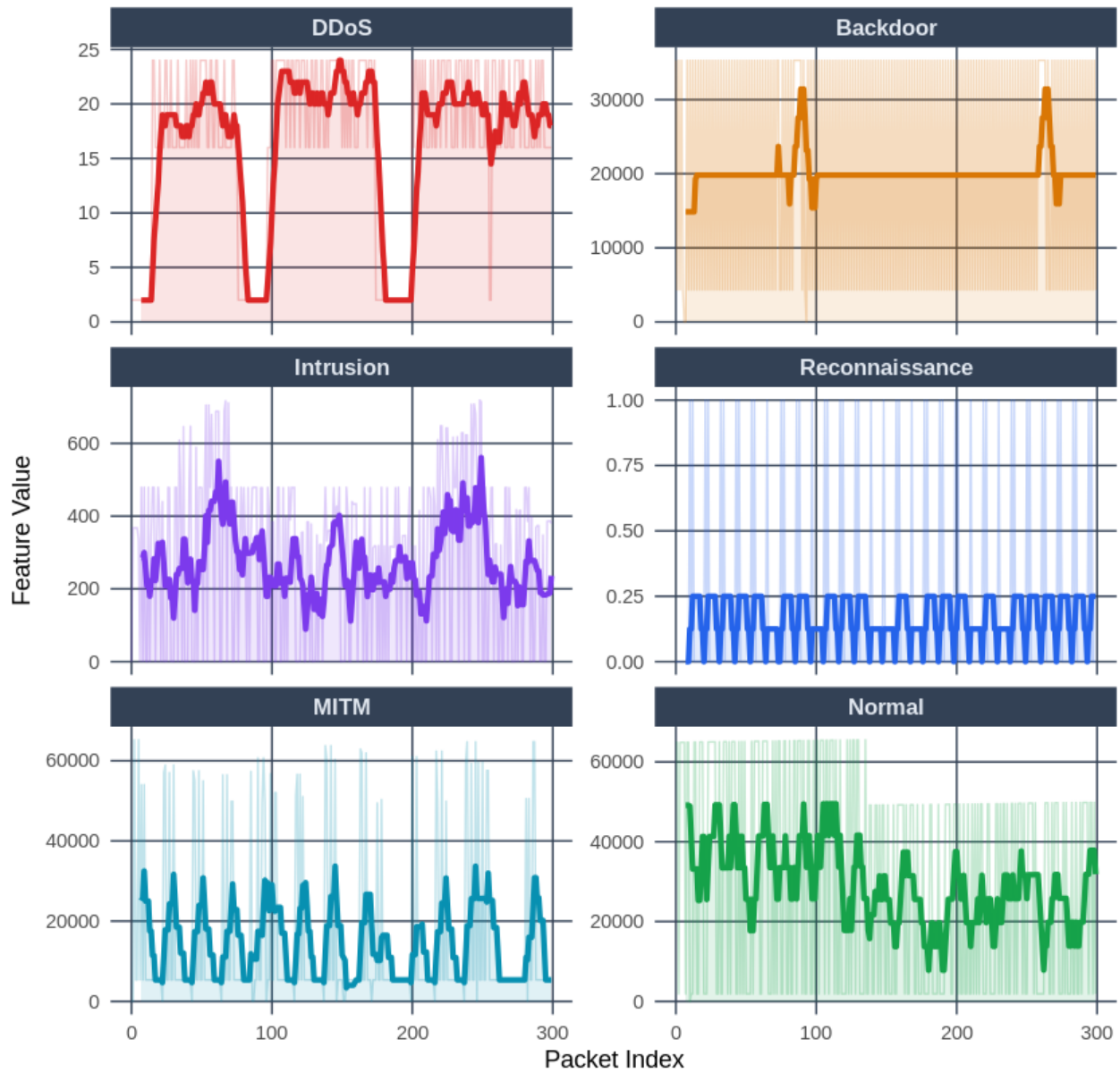


Figure 4.3: Patterns of the most discriminative feature per attack class.

22, interrupted by two sharp drops to 0 (around indices 80 –120 and 180 –210), forming a regular sawtooth pattern. The **Backdoor** class exhibits a very stable baseline around 20,000, with two narrow spikes close to 30,000 at approximately indices 85 and 235. The **Reconnaissance** class follows a quasi-binary behavior, with low values ranging from 0.10 to 0.25. It shows frequent peaks up to 0.25, corresponding to port scanning activity, as well as short stabilization periods around 0.125 after a few peak cycles. The **Normal** traffic fluctuates moderately between 30,000 and 40,000 with an overall stable trend, making it the smoothest reference behavior. The **MITM** class exhibits strong oscillations between approximately 5,000 and 30,000, with occasional peaks exceeding 60,000, reflecting continuous interception activity. Finally, the **Intrusion** class is the most irregular, with no clear structure or periodicity. It shows low values (around 30 to 100) and occasional peaks exceeding 460, which explains the classification difficulty and its lower F1-score.

The excellent AUC of 0.9986, XGBoost allows the use of a conservative threshold, achieving less than 1% false alarms while maintaining detection above 95%. This directly satisfies the Neyman - Pearson constraint introduced in Chapter 3 and supports the mechanism described in Section 4.7. XGBoost is therefore retained as the reference model.

The next Section 4.6 first evaluates its performance on the simulated dataset, then analyzes its adaptation capability when retrained on different mixtures of Edge-IIoTset and simulated data under several proportions.

4.6 Model Performance on Simulated Data and Mixed Data

Training (Edge-IIoTset / Simulated)	Accuracy	F1-Score	AUC	Inference (ms)
100% Sim	0.8619	0.8611	0.9773	0.0220
80% Edge / 20% Sim	0.9524	0.9550	0.9981	0.0196
75% Edge / 25% Sim	0.9500	0.9524	0.9981	0.0169
50% Edge / 50% Sim	0.9483	0.9507	0.9980	0.0150
25% Edge / 75% Sim	0.9461	0.9471	0.9980	0.0165
20% Edge / 80% Sim	0.9358	0.9367	0.9972	0.0176
100% Edge / 100% Sim	0.9513	0.9532	0.9981	0.0148

Table 4.3: Performance of XGBoost on simulated data and mixed training configurations.

To evaluate the model's adaptability, XGBoost was trained on different mixtures of the two datasets (simulated data and Edge-IIoTset), ranging from 0% to 100% simulated samples. The same preprocessing pipeline and hyperparameters as in Section 3.4.2 were used. The results are presented in Table 4.3. Even with 80% simulated data, the accuracy remains above 0.93, the AUC exceeds 0.997 and the inference time never goes beyond 0.02 ms. In other words, an XGBoost model trained from scratch on a mixed dataset adapts quickly to the new environment.

This shows that the gap is due to a distribution shift, not poor simulated data and can be reduced by adding a small amount of Edge-IloTset samples. Training locally on the same data already achieves an accuracy of 0.86, while adding only 20% Edge-IloTset data restores performance close to the original model. The deployment strategy is therefore straightforward.

4.7 Active Prevention System

Once trained, the XGBoost model is integrated into a prevention mechanism capable of reacting before the end of an OTA transaction, relying on the attack signatures learned from the Edge-IloTset dataset.

Alert Levels and Associated Actions

Rather than responding in a binary way, either blocking or allowing traffic, the system adapts its reaction according to the confidence level of the model. Thus, four severity levels are defined from the classifier output probability. The more confident XGBoost is that a session is malicious, the stronger the triggered countermeasure becomes.

Level	Probability threshold	Triggered action
Level 0	$p \leq 0.50$	No action
Level 1	$0.50 < p \leq 0.70$	Alert logged
Level 2	$0.70 < p \leq 0.85$	Administrator notification
Level 3	$0.85 < p \leq 0.95$	IP blocking + Re-authentication
Level 4	$p > 0.95$	Immediate firmware rollback

Table 4.4: Alert levels and triggered actions.

The logic is intuitive. When the probability barely exceeds 0,50 (Level 1), the model remains uncertain. Thus, simply logging the alert for later analysis is sufficient, without interrupting the update process.

When the confidence reaches between 0,70 and 0,85 (Level 2), uncertainty decreases and an administrator can be alerted.

Between 0,85 and 0,95 (Level 3), the model confidence is high, allowing stricter measures such as temporary IP blocking without stopping the update. Beyond 0,95 (Level 4), the model is almost certain that the activity is malicious, so the firmware is immediately rolled back.

This final threshold is made reliable by the exceptional AUC of the model (0,9986). As shown by the ROC curve [6] and the Neyman-Pearson framework [14], setting the threshold at 0,95 makes it possible to trigger a strong alarm while maintaining the false alarm rate below 1%, with more than 95% of real attacks detected. In practice, this means that a rollback will almost never interrupt a legitimate update.

The threshold values (0,50, 0,70, 0,85, 0,95) were defined by observing the probability distribution on the validation set, so that each alert level receives a reasonable number of detections. They provide a practical starting point that can be adjusted depending on the risk tolerance of each deployment.

Figure 4.4 confirms this consistency. The vast majority of detections on the test set fall into Level 4, demonstrating that the model most often makes decisions with near-absolute confidence.

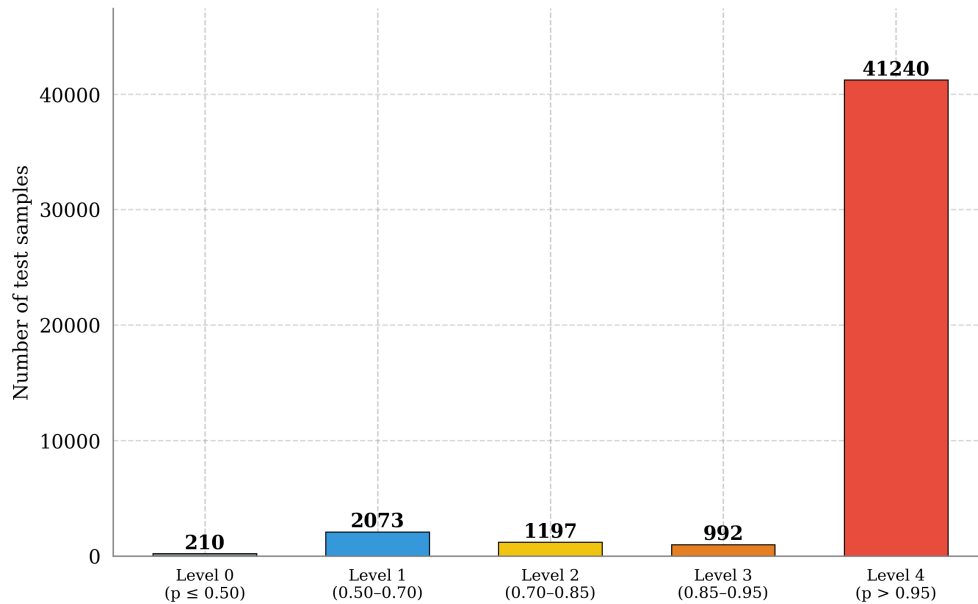


Figure 4.4: Distribution of severity levels on the test set.

5. Conclusion and Perspectives

This thesis aimed to design and evaluate a complete detection and prevention pipeline for attacks targeting IoT firmware OTA updates, leveraging both machine learning and deep learning techniques. Experiments conducted on the public Edge-IloTset dataset quickly highlighted the superiority of XGBoost, which emerged as the best trade-off between accuracy and speed. With an accuracy of 0.9676, an AUC of 0.9986 and an inference time of only 0.0135 ms per sample, it outperformed all other tested classifiers, including deep learning models and attention-based ensemble. Its very low computational footprint makes it particularly suitable for deployment on resource-constrained IoT devices.

In a second phase, a simulation framework was used to generate a dataset of 3,600 enriched network sessions covering ARP, ICMP, DNS and MQTT protocols. When trained exclusively on this synthetic data, XGBoost achieved an accuracy of 0.8619, already demonstrating strong robustness. More importantly, experiments that combine synthetic data with Edge-IloTset show that incorporating only 20% of real samples is sufficient to push accuracy beyond 0.95. This result is encouraging, as it demonstrates that local adaptation of the model to a new network environment can be achieved easily without requiring large-scale data collection.

These performances served as the foundation for a four-level alert system driven by model confidence, enabling a graduated response where the highest level triggers an immediate firmware rollback upon confirmed threat detection. The excellent AUC value confirms that a decision threshold above 0.95 can be enforced while keeping the false alarm rate below 1%, which is essential to avoid unnecessary disruption of network operations.

To extend this work, the most immediate and natural perspective is to reproduce the entire experimental pipeline in a real IoT environment with physical devices and authentic network conditions, in order to capture protocol noise that simulation can only approximate. Such an operational deployment would provide a decisive validation step and open the way toward real-world integration. Additionally, exploring recent deep learning architectures specifically designed for tabular data remains a promising research direction with the aim of narrowing the performance gap that still exists between these models and tree-based methods.

References

- [1] Muhanna Ahmed Ali and Salah Alawi Hussein Al-Sharafi. Intrusion detection in iot networks using machine learning and deep learning approaches for mitm attack mitigation. *Discover Internet of Things*, 5(1):48, 2025.
- [2] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [3] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16: 321–357, 2002.
- [4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [5] Maria Júlia Berriel de Sousa, Luis Fernando Gomez Gonzalez, Erick Mascagni Ferdinando, and Juliana Freitag Borin. Over-the-air firmware update for iot devices on the wild. *Internet of Things*, 19:100578, 2022.
- [6] Tom Fawcett. Roc analysis in pattern recognition. *Pattern Recognition Letters*, 8:861–874, 2005.
- [7] Mohamed Amine Ferrag, Othmane Friha, Djallel Hamouda, Leandros Maglaras, and Helge Janicke. Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning. *IEEE Access*, 10:40281–40306, 2022.
- [8] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in neural information processing systems*, 35:507–520, 2022.
- [9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [10] Muhammad Ibrahim, Andrea Continella, and Antonio Bianchi. Aot-attack on things: A security analysis of iot firmware updates. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pages 1047–1064. IEEE, 2023.
- [11] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [12] Erik Larsen, Korey MacVittie, and John Lilly. A survey of machine learning algorithms for detecting malware in iot firmware. *arXiv preprint arXiv:2111.02388*, 2021.
- [13] Agu Edward Onyebueke, Addakenjo Ali David, and Stephen Munu. Network intrusion detection system using xgboost and random forest algorithms. *Asian J. Pure Appl. Math*, 5 (1):321–335, 2023.

-
- [14] Clayton Scott and Robert Nowak. A neyman-pearson approach to statistical learning. *IEEE Transactions on Information Theory*, 51(11):3806–3819, 2005.
 - [15] Vladimir N Vapnik. The nature of statistical learning. *(No Title)*, 1998.
 - [16] Sami Yaras and Murat Dener. lot-based intrusion detection system using new hybrid deep learning algorithm. *Electronics*, 13(6):1053, 2024.

APPENDICES

TShark Commands for Feature Extraction

The same TShark command structure was used to extract 38 raw fields from every .pcap file, both for the original Edge-IloTset dataset and for the simulated sessions.

Packet-level extraction:

```
tshark -r session.pcap -T fields \  
-e arp.opcode -e arp.hw.size \  
-e icmp.checksum -e icmp.seq_le -e icmp.transmit_timestamp \  
-e http.content_length -e http.response \  
-e tcp.ack -e tcp.ack_raw -e tcp.checksum \  
-e tcp.connection.fin -e tcp.connection.rst \  
-e tcp.connection.syn -e tcp.connection.synack \  
-e tcp.dstport -e tcp.flags -e tcp.flags.ack \  
-e tcp.len -e tcp.options -e tcp.payload \  
-e tcp.seq -e tcp.srcport \  
-e udp.port -e udp.stream -e udp.time_delta \  
-e dns.qry.name -e dns.qry.name.len -e dns.qry.qu \  
-e dns.retransmission -e dns.retransmit_request \  
-e mqtt.conflag.cleansess -e mqtt.conflags \  
-e mqtt.hdrflags -e mqtt.len -e mqtt.msgtype \  
-e mqtt.proto_len -e mqtt.topic_len -e mqtt.ver
```

Forcing HTTP dissection on port 5000:

```
tshark -o http.tcp.port:5000 -r session.pcap -T fields -e http.response ...
```

Capture command (loopback interface):

```
tshark -i 9 -w session.pcap
```

The raw extracted values were aggregated per session (sum, mean, or first value) as described in Section [3.5.3](#)

Simulation Commands All commands below were executed in the OTA_simulation_IOT directory with the Python virtual environment activated.

Services

- OTA server (Flask, port 5000):

```
python server.py
```

- **Malicious server (Flask, port 5001):**

```
python server_backdoor.py
```

- **MQTT broker (Mosquitto):**

```
"C:\Program Files\mosquitto\mosquitto.exe"
```

- **mitmproxy (MITM):**

```
mitmproxy -s modify_firmware.py -p 8080 --listen-host 127.0.0.1  
--mode reverse:http://127.0.0.1:5000
```

Attack and Client Scripts

- **Normal client:** `python client_normal.py`
- **DDoS (SYN flood):** `python ddos_attack.py`
- **Intrusion (SQL injection):** `python intrusion_attack.py`
- **Backdoor client:** `python client_backdoor.py`
- **MITM client:** `python client_mitm.py`
- **Reconnaissance (nmap):** `nmap -sT -Pn -p 1-1000 127.0.0.1`

Protocol Enrichment

- **ARP:** `python arp_traffic.py`
- **ICMP:** `python icmp_traffic.py`
- **DNS:** `python dns_traffic_loopback.py`
- **MQTT:** `python mqtt_traffic.py`

All scripts were executed in the `C:\Users\papem\Desktop\OTA_simulation_IOT` directory with the Python virtual environment activated. All sessions were generated automatically by the script `generate_sessions.py`, which executes the above commands in loops for each scenario while simultaneously capturing traffic with TShark. The complete script is available on request.

Confusion Matrices



Figure 5.1: Confusion matrices of all models on the Edge-IIoTset test set.

Training Curves

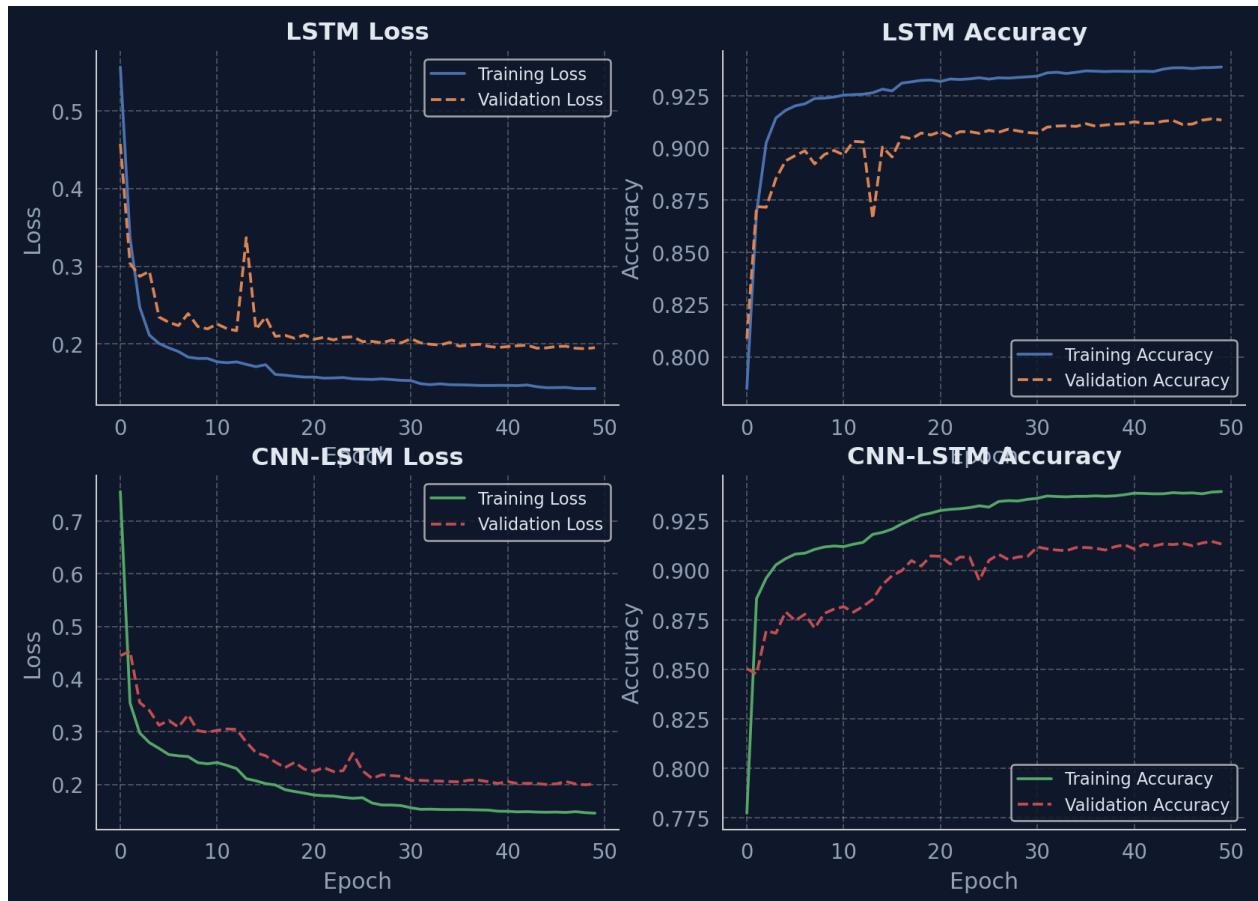


Figure 5.2: Training and validation loss and accuracy for LSTM and CNN-LSTM.